

Implementation of Interpolating Piecewise Quadratic Bezier Curves with G1 Continuity

HARSH PRITEHSKUMAR MISTRY, Indraprastha Institute of Information Technology, India

This project focuses on implementing interpolating piecewise quadratic Bezier curves with G1 continuity in C++ using OpenGL and ImGui for rendering. The program dynamically updates the curves based on user interaction with control points.

ACM Reference Format:

Harsh Pritheskumar Mistry. 2024. Implementation of Interpolating Piecewise Quadratic Bezier Curves with G1 Continuity. 1, 1 (September 2024), 3 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

This project focuses on implementing interpolating piecewise quadratic Bezier curves with G1 continuity in C++ using OpenGL and ImGui for rendering. The program dynamically updates the curves based on user interaction with control points.

2 DESIGN STRATEGY

2.1 Overview

The design strategy for implementing piecewise quadratic Bezier curves with C1 continuity involves calculating appropriate control points to guarantee smooth transitions between curve segments. The process includes deciding the C1 control points, computing the Bezier curve points, and managing user interactions for real-time updates.

2.2 Algorithm for C1 Control Points

The `calculateC1controlpoint()` function computes the C1 control points required to maintain G1 continuity. The algorithm proceeds as follows:

- (1) **Clear Previous Control Points:** The function starts by clearing the `c1controlpoints` vector to remove any previously computed values.
- (2) **Check Minimum Segments:** Ensure there are at least two segments (i.e., at least six control points) before proceeding with the calculations.
- (3) **Compute Control Points:**
 - For the first segment, compute the new control point by applying an offset from the initial segment's endpoints.
 - For subsequent segments, reflect the previous control point across the shared endpoint to align the tangents smoothly.

Author's address: Harsh Pritheskumar Mistry, harsh22200@iiitd.ac.in, Indraprastha Institute of Information Technology, Delhi, Delhi, India.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2024/9-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

- (4) **Store Results:** The calculated C1 control points are stored in the `c1controlpoints` vector with an additional z-coordinate set to 0.

2.3 Bezier Curve Calculation

The `calculatePiecewiseQuadraticBezier()` function generates the points for the piecewise quadratic Bezier curve as follows:

- (1) **Clear Previous Data:** The `quadraticBezier` vector is cleared to prepare for the new set of curve points.
 (2) **Sample Points:** For each segment, compute the curve points using the quadratic Bezier formula:

$$B(t) = (1 - t)^2 x_0 + 2(1 - t)t x_1 + t^2 x_2 \quad (1)$$

where t is sampled at regular intervals.

- (3) **Store Results:** The computed curve points are stored in the `quadraticBezier` vector.

2.4 Data Structures

- **Control Points:** Managed using `std::vector<float>`.
- **Bezier Curve Points:** Stored in `std::vector<float>` for both C1 control points and quadratic Bezier curve points.

3 IMPLEMENTATION DETAILS

3.1 User Interaction

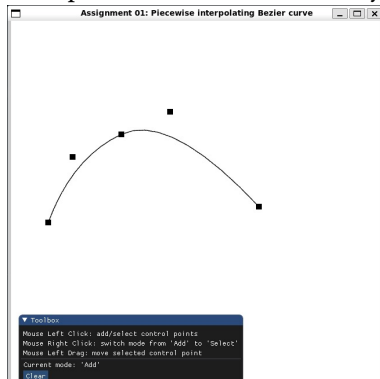
The user can add control points by clicking on the canvas. Right-clicking stops the addition of points. The program calculates the Bezier curves and enforces C1/G1 continuity while the control points are being added. Control points can be moved by clicking and dragging after the points are finalized. The curves are recalculated dynamically based on the updated positions.

3.2 Drawing the Curve

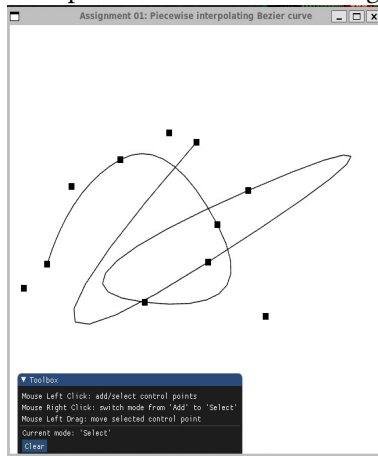
The program uses OpenGL to render the points and curves. VAO (Vertex Array Object) and VBO (Vertex Buffer Object) are employed to store and update the vertex data. The curves are drawn using `glDrawArrays()` in a line strip mode.

4 SCREENSHOTS

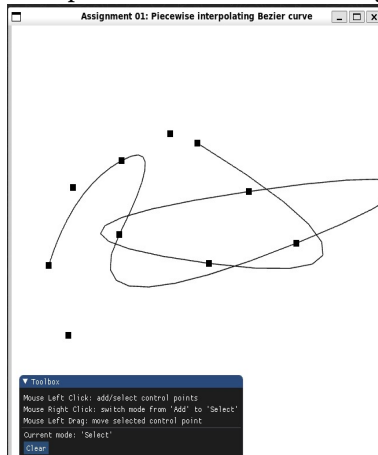
- Example 1: Initial curve created by adding three control points.



- Example 2: The curve after adding more points and ensuring G1 continuity.



- Example 3: The curve after moving a control point to demonstrate dynamic updates.



5 CONCLUSION

The implementation successfully draws interpolating piecewise quadratic Bezier curves with G1 continuity. The program supports real-time user interaction to add and modify control points. Managing G1 continuity dynamically during user interactions was a key challenge. Possible improvements include adding features like interactive tangent modification or allowing the user to change curve properties.